

- GORM04** Gorman, M. *Understanding the Linux Virtual Memory Manager*. Upper Saddle River, NJ: Prentice Hall, 2004.
- JACO98a** Jacob, B., and Mudge, T. "Virtual Memory: Issues of Implementation." *Computer*, June 1998.
- JACO98b** Jacob, B., and Mudge, T. "Virtual Memory in Contemporary Microprocessors." *IEEE Micro*, August 1998.
- VAHA96** Vahalia, U. *UNIX Internals: The New Frontiers*. Upper Saddle River, NJ: Prentice Hall, 1996.



Animations

A set of animation that illustrates concepts from this chapter is available at the Premium Web site. The reader is encouraged to view the animations to reinforce concepts from this chapter.

8.9 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

associative mapping demand paging external fragmentation fetch policy frame hash table hashing internal fragmentation locality	page page fault page placement policy page replacement policy page table paging prepaging real memory resident set	resident set management segment segment table segmentation slab allocation thrashing translation lookaside buffer virtual memory working set
--	--	--

Review Questions

- 8.1.** What is the difference between simple paging and virtual memory paging?
- 8.2.** Explain thrashing.
- 8.3.** Why is the principle of locality crucial to the use of virtual memory?
- 8.4.** What elements are typically found in a page table entry? Briefly define each element.
- 8.5.** What is the purpose of a translation lookaside buffer?
- 8.6.** Briefly define the alternative page fetch policies.
- 8.7.** What is the difference between resident set management and page replacement policy?
- 8.8.** What is the relationship between FIFO and clock page replacement algorithms?
- 8.9.** What is accomplished by page buffering?
- 8.10.** Why is it not possible to combine a global replacement policy and a fixed allocation policy?
- 8.11.** What is the difference between a resident set and a working set?
- 8.12.** What is the difference between demand cleaning and precleaning?

Problems

8.1. Suppose the page table for the process currently executing on the processor looks like the following. All numbers are decimal, everything is numbered starting from zero, and all addresses are memory byte addresses. The page size is 1,024 bytes.

Virtual page number	Valid bit	Reference bit	Modify bit	Page frame number
0	1	1	0	4
1	1	1	1	7
2	0	0	0	—
3	1	0	0	2
4	0	0	0	—
5	1	0	1	0

- a. Describe exactly how, in general, a virtual address generated by the CPU is translated into a physical main memory address.
- b. What physical address, if any, would each of the following virtual addresses correspond to? (Do not try to handle any page faults, if any.)
 - (i) 1,052
 - (ii) 2,221
 - (iii) 5,499

8.2. Consider the following program.

```
#define Size 64
int A[Size; Size], B[Size; Size], C[Size; Size];
int register i, j;

for (j = 0; j< Size; j ++ )
    for (i = 0; i< Size; i++)
        C[i; j] = A[i; j] + B[i; j];
```

Assume that the program is running on a system using demand paging and the page size is 1 Kilobyte. Each integer is 4 bytes long. It is clear that each array requires a 16-page space. As an example, A[0, 0]-A[0, 63], A[1, 0]-A[1, 63], A[2, 0]-A[2, 63], and A[3, 0]-A[3, 63] will be stored in the first data page. A similar storage pattern can be derived for the rest of array A and for arrays B and C. Assume that the system allocates a 4-page working set for this process. One of the pages will be used by the program and three pages can be used for the data. Also, two index registers are assigned for i and j (so, no memory accesses are needed for references to these two variables).

- a. Discuss how frequently the page fault would occur (in terms of number of times $C[i, j] = A[i, j] + B[i, j]$ are executed).
 - b. Can you modify the program to minimize the page fault frequency?
 - c. What will be the frequency of page faults after your modification?
- 8.3. a. How much memory space is needed for the user page table of Figure 8.3?
b. Assume you want to implement a hashed inverted page table for the same addressing scheme as depicted in Figure 8.3, using a hash function that maps the 20-bit page number into a 6-bit hash value. The table entry contains the page number, the frame number, and a chain pointer. If the page table allocates space for up to 3 overflow entries per hashed entry, how much memory space does the hashed inverted page table take?

- 8.4.** Consider the following string of page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. Complete a figure similar to Figure 8.14, showing the frame allocation for:
- FIFO (first-in-first-out)
 - LRU (least recently used)
 - Clock
 - Optimal (assume the page reference string continues with 1, 2, 0, 1, 7, 0, 1)
 - List the total number of page faults and the miss rate for each policy. Count page faults only after all frames have been initialized.
- 8.5.** A process references five pages, A, B, C, D, and E, in the following order:
- A; B; C; D; A; B; E; A; B; C; D; E

Assume that the replacement algorithm is first-in-first-out and find the number of page transfers during this sequence of references starting with an empty main memory with three page frames. Repeat for four page frames.

- 8.6.** A process contains eight virtual pages on disk and is assigned a fixed allocation of four page frames in main memory. The following page trace occurs:

1, 0, 2, 2, 1, 7, 6, 7, 0, 1, 2, 0, 3, 0, 4, 5, 1, 5, 2, 4, 5, 6, 7, 6, 7, 2, 4, 2, 7, 3, 3, 2, 3

- Show the successive pages residing in the four frames using the LRU replacement policy. Compute the hit ratio in main memory. Assume that the frames are initially empty.
 - Repeat part (a) for the FIFO replacement policy.
 - Compare the two hit ratios and comment on the effectiveness of using FIFO to approximate LRU with respect to this particular trace.
- 8.7.** In the VAX, user page tables are located at virtual addresses in the system space. What is the advantage of having user page tables in virtual rather than main memory? What is the disadvantage?
- 8.8.** Suppose the program statement

```
for (i = 1; i ≤ n; i++)
    a[i] = b[i] + c[i];
```

is executed in a memory with page size of 1,000 words. Let $n = 1,000$. Using a machine that has a full range of register-to-register instructions and employs index registers, write a hypothetical program to implement the foregoing statement. Then show the sequence of page references during execution.

- 8.9.** The IBM System/370 architecture uses a two-level memory structure and refers to the two levels as segments and pages, although the segmentation approach lacks many of the features described earlier in this chapter. For the basic 370 architecture, the page size may be either 2 Kbytes or 4 Kbytes, and the segment size is fixed at either 64 Kbytes or 1 Mbyte. For the 370/XA and 370/ESA architectures, the page size is 4 Kbytes and the segment size is 1 Mbyte. Which advantages of segmentation does this scheme lack? What is the benefit of segmentation for the 370?
- 8.10.** Assuming a page size of 4 Kbytes and that a page table entry takes 4 bytes, how many levels of page tables would be required to map a 64-bit address space, if the top-level page table fits into a single page?
- 8.11.** Consider a system with memory mapping done on a page basis and using a single-level page table. Assume that the necessary page table is always in memory.
- If a memory reference takes 200 ns, how long does a paged memory reference take?
 - Now we add an MMU that imposes an overhead of 20 ns on a hit or a miss. If we assume that 85% of all memory references hit in the MMU TLB, what is the Effective Memory Access Time (EMAT)?
 - Explain how the TLB hit rate affects the EMAT.

- 8.12.** Consider a page reference string for a process with a working set of M frames, initially all empty. The page reference string is of length P with N distinct page numbers in it. For any page replacement algorithm,
- What is a lower bound on the number of page faults?
 - What is an upper bound on the number of page faults?
- 8.13.** In discussing a page replacement algorithm, one author makes an analogy with a snowplow moving around a circular track. Snow is falling uniformly on the track and a lone snowplow continually circles the track at constant speed. The snow that is plowed off the track disappears from the system.
- For which of the page replacement algorithms discussed in Section 8.2 is this a useful analogy?
 - What does this analogy suggest about the behavior of the page replacement algorithm in question?
- 8.14.** In the S/370 architecture, a storage key is a control field associated with each page-sized frame of real memory. Two bits of that key that are relevant for page replacement are the reference bit and the change bit. The reference bit is set to 1 when any address within the frame is accessed for read or write, and is set to 0 when a new page is loaded into the frame. The change bit is set to 1 when a write operation is performed on any location within the frame. Suggest an approach for determining which page frames are least-recently-used, making use of only the reference bit.
- 8.15.** Consider the following sequence of page references (each element in the sequence represents a page number):

1 2 3 4 5 2 1 3 3 2 3 4 5 4 5 1 1 3 2 5

Define the *mean working set size* after the k th reference as $s_k(\Delta) = \frac{1}{k} \sum_{t=1}^k |W(t, \Delta)|$ and define the *missing page probability* after the k th reference as $m_k(\Delta) = \frac{1}{k} \sum_{t=1}^k |F(t, \Delta)|$ where $F(t, \Delta) = 1$ if a page fault occurs at virtual time t and 0 otherwise.

- Draw a diagram similar to that of Figure 8.17 for the reference sequence just defined for the values $\Delta = 1, 2, 3, 4, 5, 6$.
 - Plot $s_{20}(\Delta)$ as a function of Δ .
 - Plot $m_{20}(\Delta)$ as a function of Δ .
- 8.16.** A key to the performance of the VSWS resident set management policy is the value of Q . Experience has shown that with a fixed value of Q for a process, there are considerable differences in page fault frequencies at various stages of execution. Furthermore, if a single value of Q is used for different processes, dramatically different frequencies of page faults occur. These differences strongly indicate that a mechanism that would dynamically adjust the value of Q during the lifetime of a process would improve the behavior of the algorithm. Suggest a simple mechanism for this purpose.
- 8.17.** Assume that a task is divided into four equal-sized segments and that the system builds an eight-entry page descriptor table for each segment. Thus, the system has a combination of segmentation and paging. Assume also that the page size is 2 Kbytes.
- What is the maximum size of each segment?
 - What is the maximum logical address space for the task?
 - Assume that an element in physical location 00021ABC is accessed by this task. What is the format of the logical address that the task generates for it? What is the maximum physical address space for the system?

- 8.18.** Consider a paged logical address space (composed of 32 pages of 2 Kbytes each) mapped into a 1-Mbyte physical memory space.
- a.** What is the format of the processor's logical address?
 - b.** What is the length and width of the page table (disregarding the "access rights" bits)?
 - c.** What is the effect on the page table if the physical memory space is reduced by half?
- 8.19.** The UNIX kernel will dynamically grow a process's stack in virtual memory as needed, but it will never try to shrink it. Consider the case in which a program calls a C subroutine that allocates a local array on the stack that consumes 10 K. The kernel will expand the stack segment to accommodate it. When the subroutine returns, the stack pointer is adjusted and this space could be released by the kernel, but it is not released. Explain why it would be possible to shrink the stack at this point and why the UNIX kernel does not shrink it.